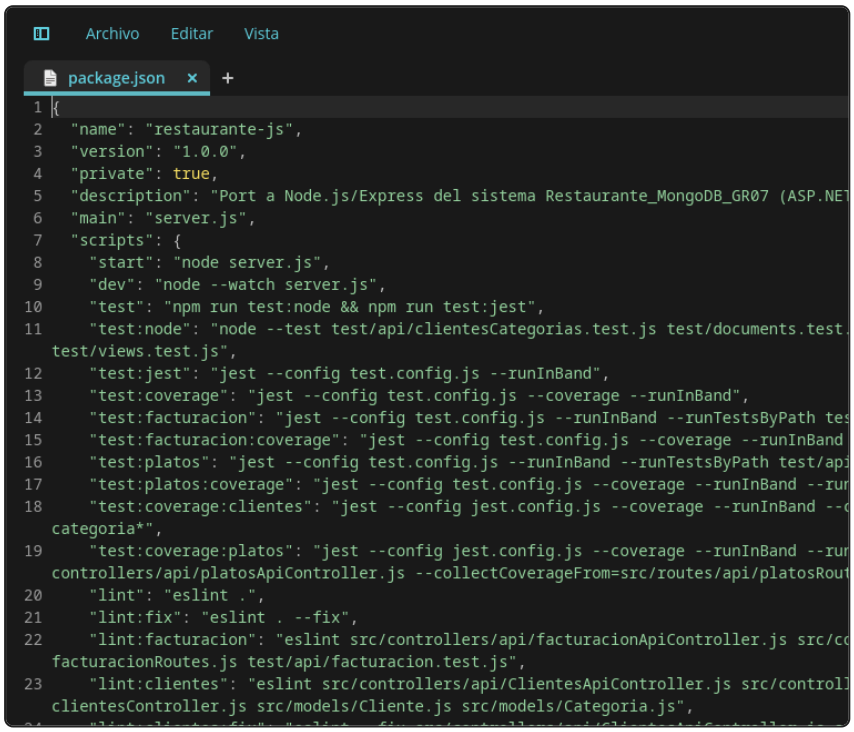
Pruebas Estáticas

Configuración de dependencias para trabajar con Jest, Supertest y ESLint

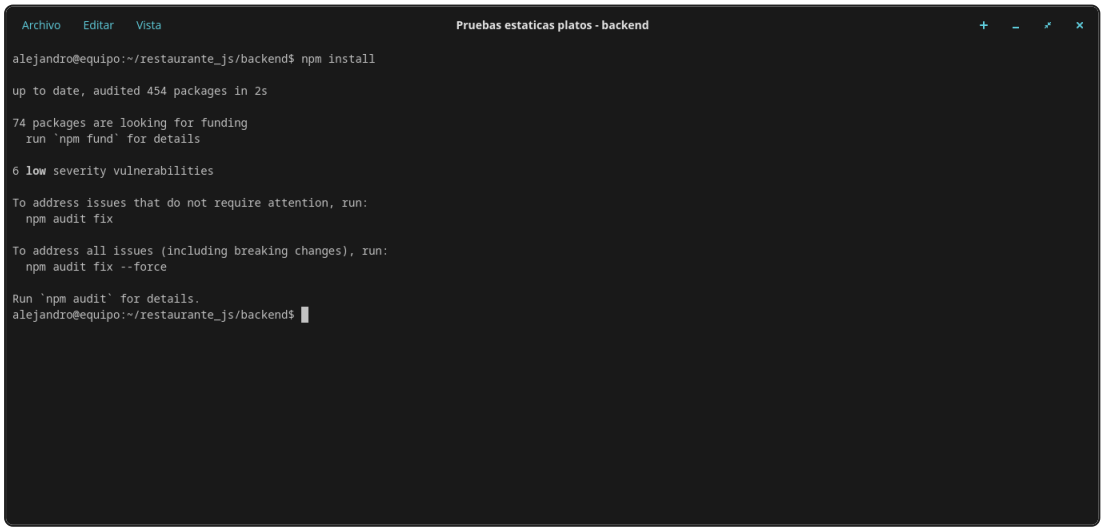
npm ESLint

El proyecto incorpora scripts específicos para el módulo de platos: lint, corrección automática, pruebas y cobertura. El código de la captura corresponde a `backend/package.json`.



`backend/package.json`: scripts `test:platos`, `test:platos:coverage` y `test:coverage:platos`.

`npm install` → descarga las dependencias necesarias dentro de la carpeta `node_modules`.

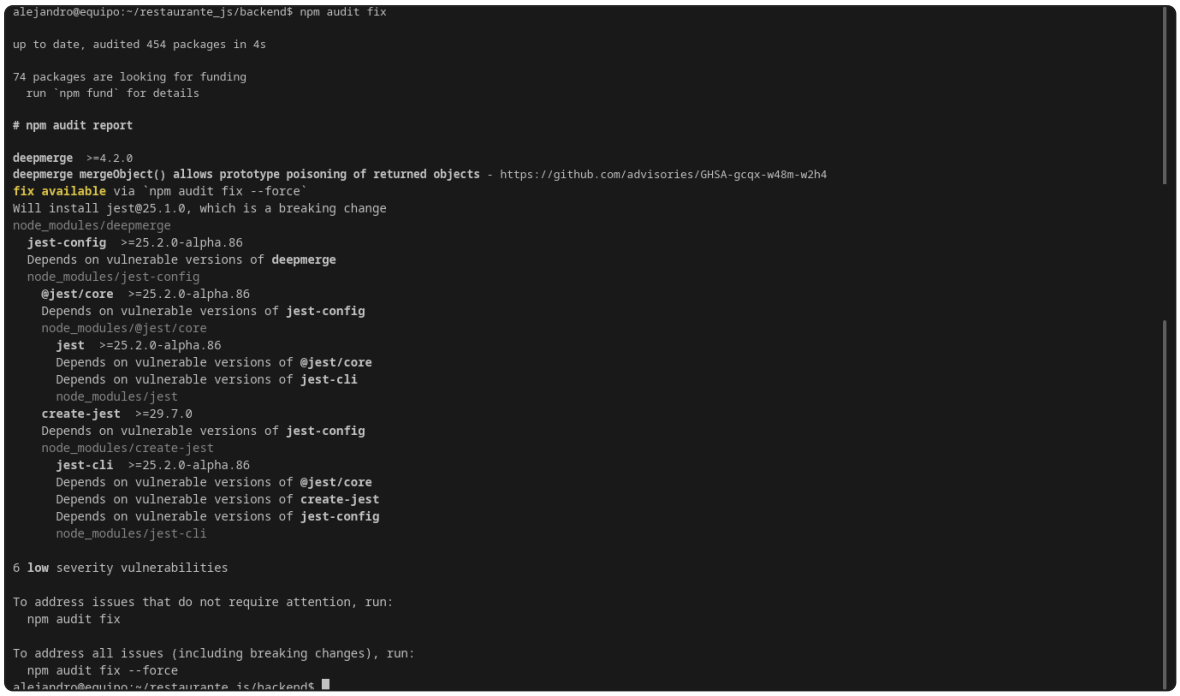


Instalación de dependencias en una terminal real del proyecto.

La instalación se ejecutó en `backend/`, el workspace que contiene Express, MongoDB, Jest y Supertest.

npm audit fix

`npm audit fix` → corrige vulnerabilidades conocidas utilizando paquetes en versiones estables.

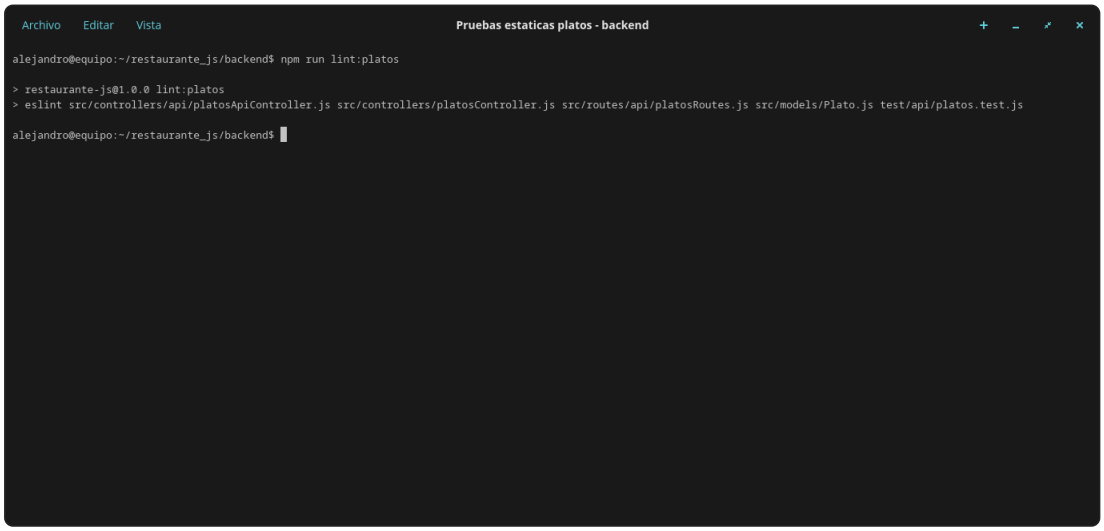


Salida completa de `npm audit fix`: 6 vulnerabilidades de severidad baja en `deepmerge`, dependencia transitiva de Jest.

No se aplicó `npm audit fix --force`: la salida advierte que instalaría `jest@25.1.0`, un cambio incompatible con la versión actual del proyecto.

npm run lint:platos

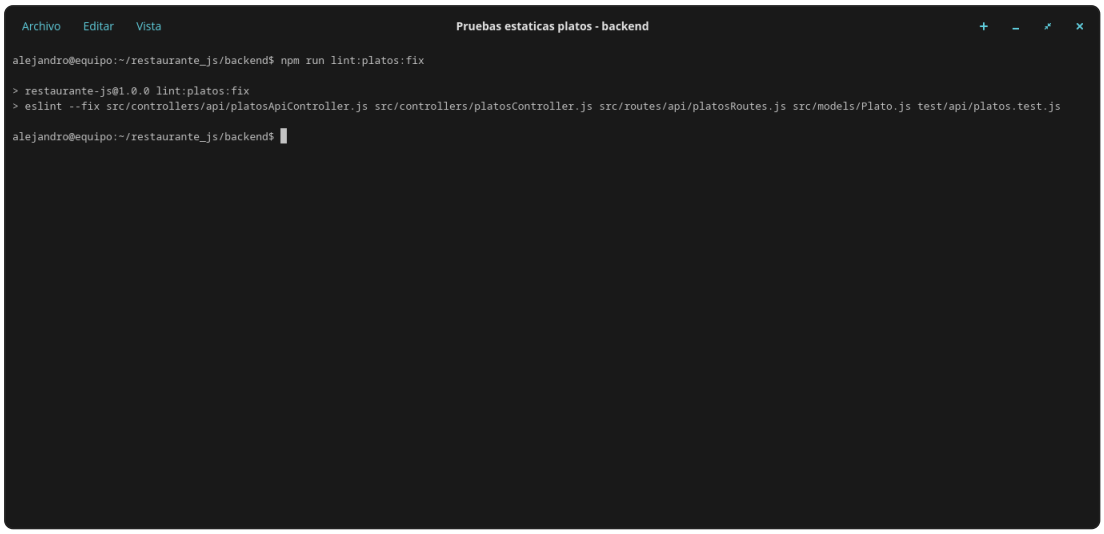
Revisión de variables sin usar, sintaxis o errores de tipiado en el controlador, las rutas, el modelo y la suite de pruebas.



`npm run lint:platos` terminó sin errores ni advertencias para los cinco archivos analizados.

npm run lint:platos:fix

Corrección automática de los problemas que ESLint puede resolver sin intervención manual.

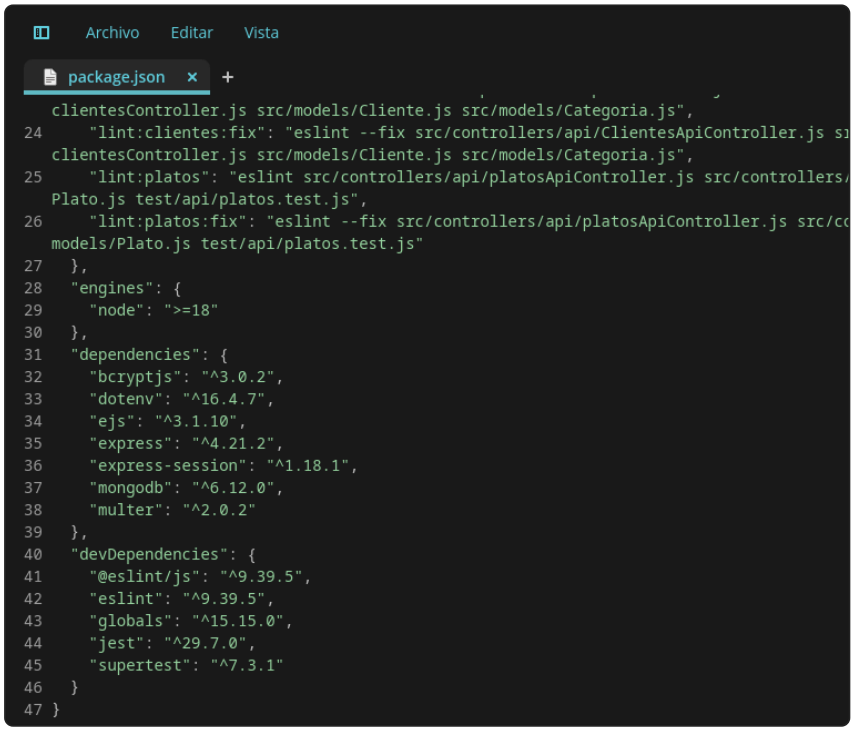


La corrección automática no reportó errores pendientes. El lint del módulo quedó limpio.

Archivos revisados: `platosApiController.js`, `platosController.js`, `platosRoutes.js`, `Plato.js` y `test/api/platos.test.js`.

Scripts de lint del módulo

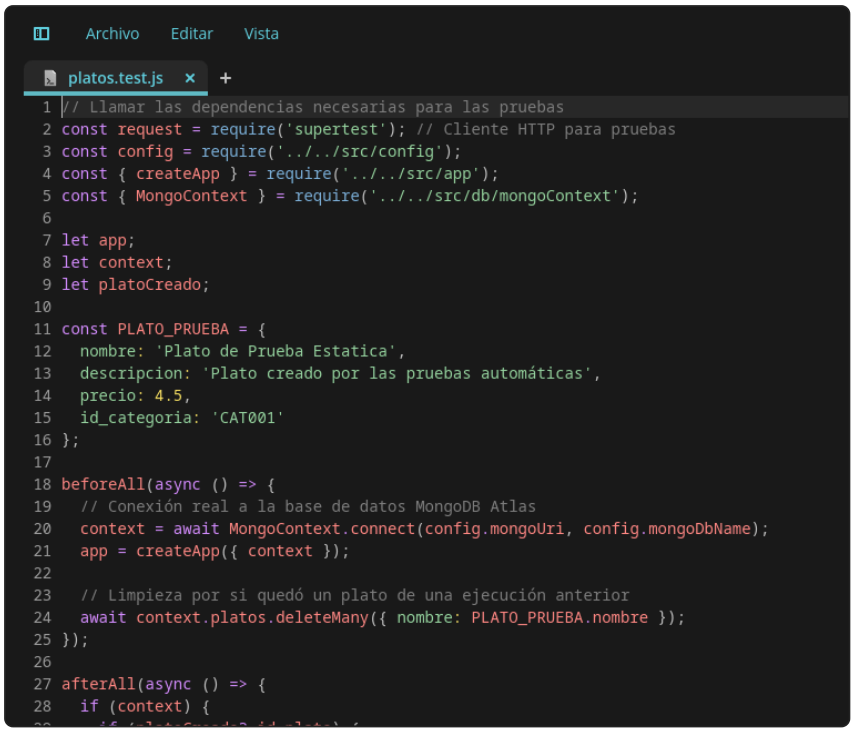
Los dos scripts siguientes concentran la verificación y la corrección automática del módulo de platos.



`backend/package.json`: definición de `lint:platos` y `lint:platos:fix` junto con las dependencias de desarrollo.

Jest + Supertest

Suite `backend/test/api/platos.test.js`: conexión real a MongoDB Atlas, CRUD completo y validación de respuestas de error.



Preparación de la suite: conexión a MongoDB Atlas, creación de la aplicación y limpieza del plato de prueba en `afterAll`.

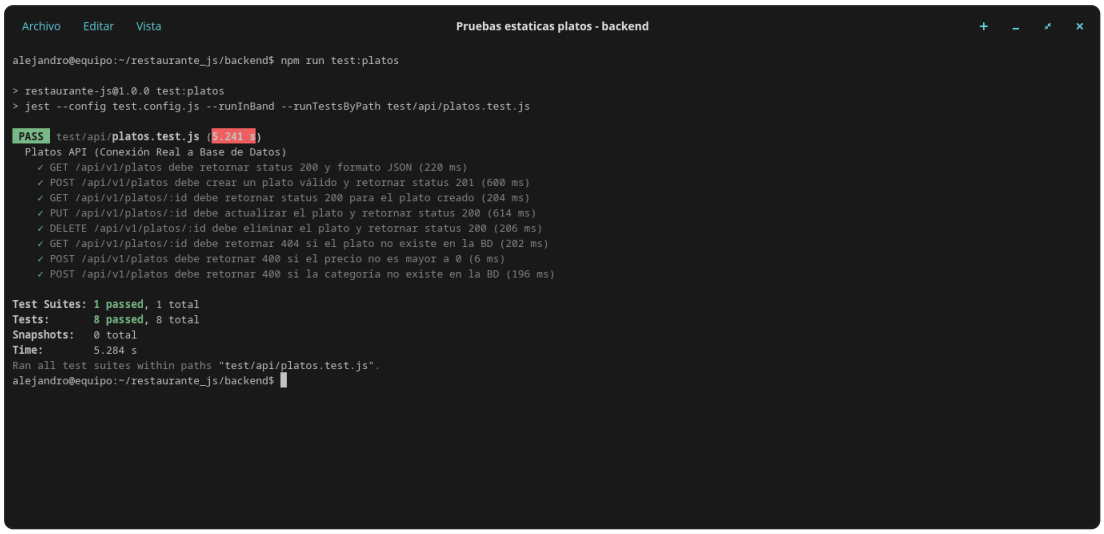
Éxitos

1. **GET** `/api/v1/platos`: devuelve **200** y una lista JSON.
2. **POST** `/api/v1/platos`: crea un plato válido y devuelve **201**.
3. **GET** `/api/v1/platos/:id`: consulta el plato recién creado y devuelve **200**.
4. **PUT** `/api/v1/platos/:id`: actualiza nombre y precio, y devuelve **200**.
5. **DELETE** `/api/v1/platos/:id`: elimina el plato y devuelve **200**.

La suite usa la base de datos real y elimina los documentos creados durante la ejecución para no dejar datos residuales.

Ejecución de la suite

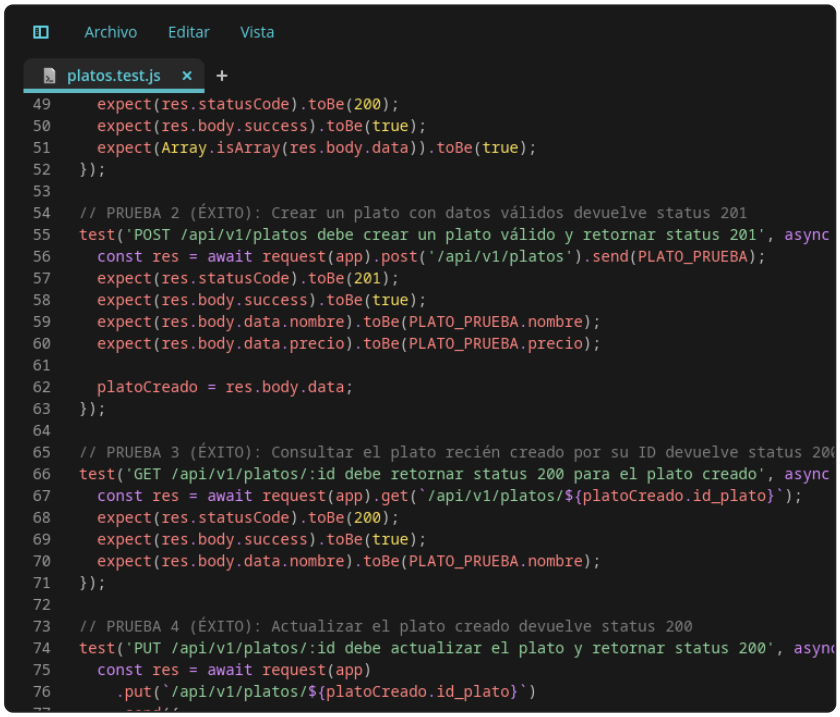
Resultado de `npm run test:platos` con Jest y Supertest.



Salida real de Jest: 8 pruebas ejecutadas, 8 aprobadas, sin fallos ni snapshots pendientes.

8/8 pruebas aprobadas. La ejecución incluye los cinco casos de éxito del CRUD y los tres casos de fracaso controlado.

Casos de éxito en el código

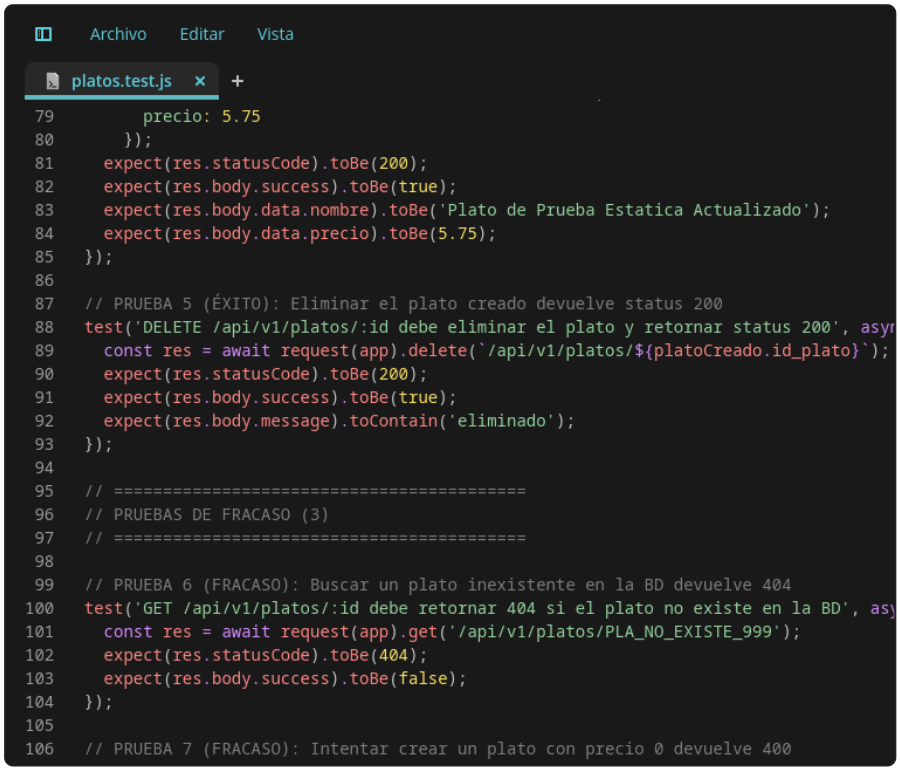


Implementación de los casos de éxito: listar, crear, consultar por ID, actualizar y eliminar.

Fracasos controlados

Las pruebas de fracaso comprueban que la API rechace las peticiones inválidas y no altere la base de datos.

1. **GET** `/api/v1/platos/PLA_NO_EXISTE_999`: devuelve **404** y `success: false`.
2. **POST** `/api/v1/platos` con `precio: 0`: devuelve **400** y el campo `precio`.
3. **POST** `/api/v1/platos` con `CAT_NO_EXISTE_999`: devuelve **400** y el campo `id_categoria`.

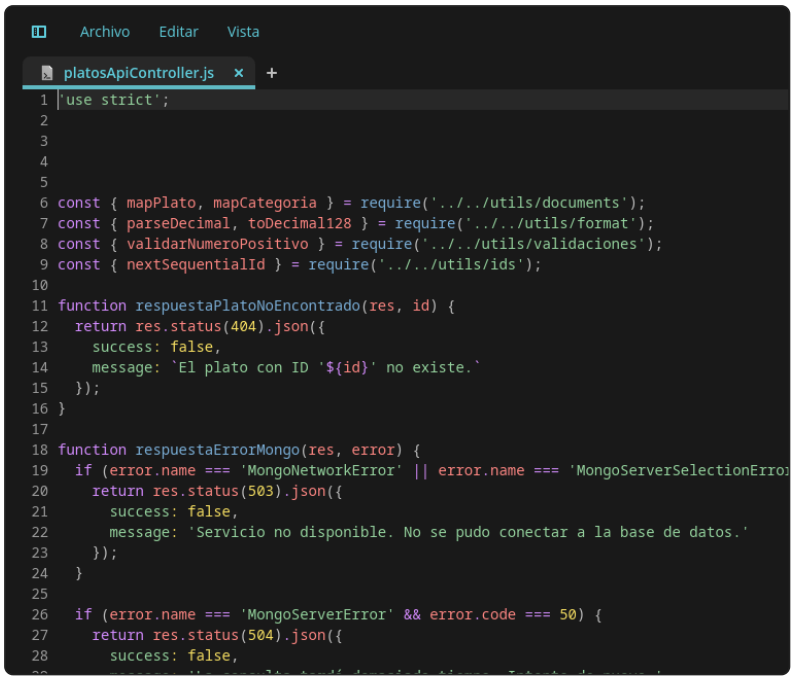


Casos de fracaso en el código: recurso inexistente y precio no positivo. La tercera prueba valida una categoría inexistente.

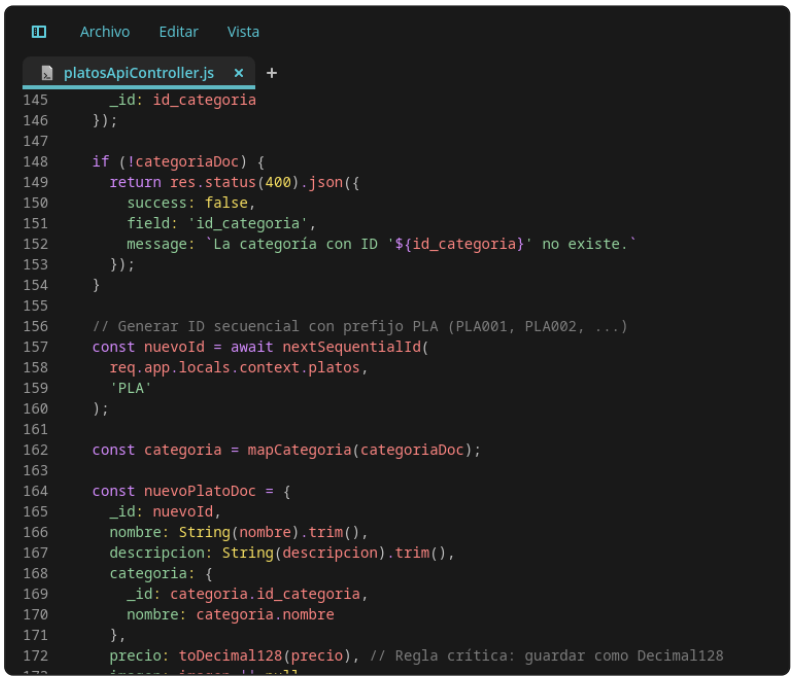
Estas pruebas también finalizaron correctamente: el resultado esperado es una respuesta de error, no una excepción.

Controlador de platos

El controlador implementa las operaciones CRUD y las reglas de negocio del módulo: validación de campos, precio positivo y existencia de la categoría.



Inicio de `platosApiController.js`: utilidades de mapeo, respuestas de error de MongoDB y consulta de platos.

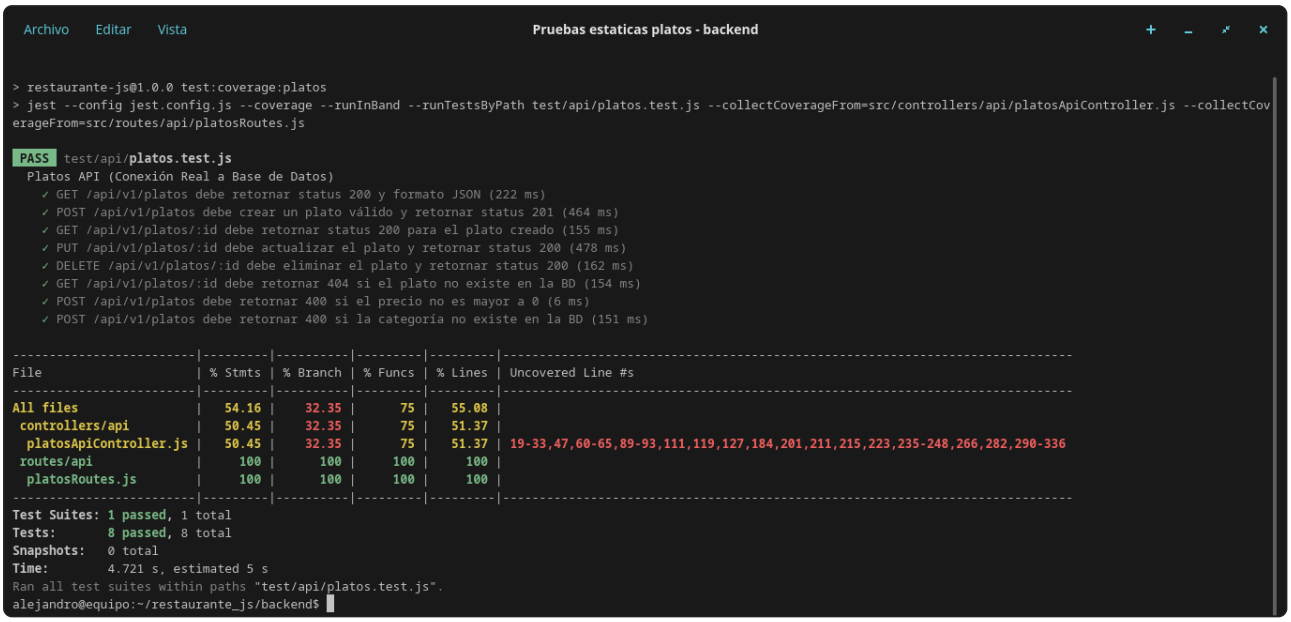


Validaciones de `crearPlato`: el precio debe ser mayor a 0 y la categoría debe existir en la base de datos.

Rutas cubiertas: `GET /api/v1/platos`, `GET /api/v1/platos/:id`, `POST /api/v1/platos`, `PUT /api/v1/platos/:id` y `DELETE /api/v1/platos/:id`.

Cobertura

Resultado de `npm run test:coverage:platos`, limitado al controlador y a las rutas del módulo de platos.



Reporte de cobertura de Jest: 54.16% de sentencias y 55.08% de líneas en el conjunto analizado; las rutas alcanzan 100%.

Resumen de cobertura

Archivo / conjunto	% Stmts	% Branch	% Funcs	% Lines
All files	54.16	32.35	75.00	55.08
`platosApiController.js`	50.45	32.35	75.00	51.37
`platosRoutes.js`	100.00	100.00	100.00	100.00

Las líneas sin cubrir corresponden principalmente a ramas de error de MongoDB y a la función auxiliar `calcularCaloriasAlergenos`, que no forma parte del CRUD probado en esta suite.